

Lab Sheet: Polymorphism (Overloading & Overriding)

Course: Introduction to Programming Language II (Java)

Topic: Compile-time Polymorphism (Method Overloading), Runtime Polymorphism (Method Overriding), Dynamic Dispatch

Level: Beginner to Intermediate

Duration: 3 hours

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU

1. Objectives

By the end of this lab, students will be able to:

- Understand the concept of Polymorphism in OOP.
- Implement Compile-time Polymorphism using Method Overloading.
- Implement Runtime Polymorphism using Method Overriding.
- Differentiate between Method Overloading and Method Overriding.
- Use constructor overloading effectively.
- Apply dynamic method dispatch in inheritance hierarchies.
- Use the `@Override` annotation correctly.

2. Introduction to Polymorphism

Polymorphism means ****many forms****. In Java, it allows one interface to be used for a general class of actions.

There are two types of Polymorphism in Java:

- **Compile-time Polymorphism (Static)** → Method Overloading
- **Runtime Polymorphism (Dynamic)** → Method Overriding + Dynamic Dispatch

3. Method Overloading (Compile-time Polymorphism)

Method overloading occurs when multiple methods in the same class have the **same name** but **different parameters** (number, type, or order).

Rules:

- Same method name
- Different parameters
- Return type does not matter for overloading
- Can be done in same class or through inheritance

4. Example 1: Method Overloading

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    String add(String a, String b) {
        return a + b;
    }
}

public class OverloadingDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(5, 10));           // int
        System.out.println(calc.add(5.5, 10.5));       // double
        System.out.println(calc.add(1, 2, 3));         // three ints
        System.out.println(calc.add("Hello", " World")); // String
    }
}
```

Output:

```
15
16.0
6
Hello World
```

5. Example 2: Constructor Overloading

```
class Student {
    String name;
    int roll;
    String department;

    Student(String name) {
        this.name = name;
        this.roll = 0;
        this.department = "Unknown";
    }

    Student(String name, int roll) {
        this.name = name;
        this.roll = roll;
        this.department = "CSE";
    }
}
```

```
Student(String name, int roll, String department) {
    this.name = name;
    this.roll = roll;
    this.department = department;
}

void display() {
    System.out.println("Name: " + name + ", Roll: " + roll + ", Dept: "
        + department);
}

public class ConstructorOverloading {
    public static void main(String[] args) {
        Student s1 = new Student("Alice");
        Student s2 = new Student("Bob", 101);
        Student s3 = new Student("Charlie", 102, "EEE");

        s1.display();
        s2.display();
        s3.display();
    }
}
```

6. Method Overriding (Runtime Polymorphism)

Method overriding occurs when a subclass provides a specific implementation of a method already defined in its superclass.

Rules:

- Same method name and parameters
- Same or covariant return type
- Must be in inheritance relationship
- Cannot override final or static methods
- Use @Override annotation (recommended)

7. Example 3: Method Overriding + Dynamic Dispatch

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }

    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }

    @Override
    void eat() {
        System.out.println("Dog eats bones");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class OverridingDemo {
    public static void main(String[] args) {
        Animal a1 = new Dog(); // Upcasting
        Animal a2 = new Cat();
        Animal a3 = new Animal();

        a1.sound(); // Dog barks
        a2.sound(); // Cat meows
        a3.sound(); // Animal makes a sound
    }
}
```

```
        a1.eat();    // Dog eats bones
    }
}
```

Output:

```
Dog barks
Cat meows
Animal makes a sound
Dog eats bones
```

Task 2.1 – Method Overloading (12 min)

a. Create a class `MathOperations` with overloaded methods named `multiply`:

- `int multiply(int a, int b)`
- `double multiply(double a, double b)`
- `int multiply(int a, int b, int c)`

Task 2.2 – Shape Area Calculator (15 min)

a. Create a class `Shape` with overloaded methods `area()` to calculate:

- Area of Circle (radius)
- Area of Rectangle (length, width)
- Area of Triangle (base, height)

Task 3.1 – Runtime Polymorphism (15 min)

- Create a class `Vehicle` with method `startEngine()`.
- Create two subclasses: `Car` and `Bike` that override `startEngine()`.
- Demonstrate dynamic dispatch using parent class reference.

Task 3.2 – Bank Account Polymorphism (15 min)

- Create `BankAccount` superclass with `withdraw(double amount)` method.
- Create `SavingsAccount` and `CurrentAccount` subclasses with different withdrawal rules.
- Show runtime polymorphism.

Keep Coding!

Polymorphism is one of the most powerful features of Object-Oriented Programming. Master it!